

Project 4 — ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief

Type: Multi-agent (5 agents: 1 Supervisor + 4 specialists) · **Stack:** Python, LangGraph, `Command` tool · **Level:** Advanced **Tagline:** A supervisor-led team — Research, Analysis, Writer, and Reviewer — that takes a topic from raw research to a fact-checked, revised final brief.

The full build spec lives in [11_multi_agent_systems/README.md](#). This file is your workspace and checklist, not a second copy of that spec.

Charter (what this project is)

Supervisor → Research Agent → Analysis Agent → Writer Agent → Reviewer Agent, with `Command`-based routing, shared state, and a limited checker/revision loop. This is the design decision the whole curriculum has been building toward: **one agent vs. many**, proven with a working system you can defend. It's also the first project built the way you'd actually build any multi-agent system at a real job: one specialist, proven simple, first — not all five agents at once.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows you splitting one complex task across specialist agents with clear, separate jobs — and just as important, shows you *knowing when that's actually worth it*, which is the harder, more senior skill.
- **Why it matters:** this is the headline project of the whole curriculum — it's what "multi-agent system" means on a resume, and exactly what most AI engineering interviews ask about ("walk me through an agent system you built, and why you designed it that way"). A reviewer sees a supervisor pattern, a limited checker/revision loop, and a written cost/speed comparison against simpler options — proof the design was a real decision, not a default.
- **When you'd build something like this at a real job:** content pipelines (research → draft → fact-check → publish), multi-step analysis tools, or any workflow with genuinely separate phases that benefit from separate specialist instructions, instead of one overloaded system prompt.
- **How it's built:** a supervisor node routing via the `Command` tool to Research/Analysis/Writer/Reviewer agents, with shared state kept deliberately organized, and a hard-limited revision loop between writer and reviewer.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The supervisor routes to the wrong specialist agent	Sharpen each specialist's description/routing rules, the same way you'd fix unclear tool choices from Doc06 — this is a prompt-writing problem, not a code problem
Writer and reviewer loop forever, never agreeing	A hard limit on revision rounds, with a clear "couldn't agree" report as the way out — the same idea as every agent loop in this curriculum
One agent's own scratch notes leak into shared state and confuse a later agent	Keep the state shape deliberately split into shared vs. per-agent fields — don't default everything to shared
Two agents redundantly call the same tool for the same sub-task	Have the supervisor track what's already been done in shared state, and route based on that, instead of letting agents blindly repeat each other's work

Setup (do this once, before Step 1)

```
cd project_4_multi_agent_system
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph python-dotenv pydantic
pip freeze > requirements.txt
```

Same core packages as Project 3 (no RAG-specific packages needed, unless your Research agent's tool uses one). Same `.env` setup as before.

Built During This Document

[11_multi_agent_systems](#)

Plan Before You Code

See [15_five_projects_index](#): before building this version, design the *same* task as a single-agent version and a sequential-agent version (see Doc11's practice exercises), and guess which one wins on cost/speed/reliability. Only then build the supervisor version.

How To Build This — Step by Step

Step 1 — One Specialist's Drafting Logic, Working Alone, No Agent Behavior

Project: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — Step 1 of 5: One Specialist's Drafting Logic, Working Alone, No Agent Behavior

What this step does: proves the Writer's core drafting logic works by itself — a plain in-prompt, out-text chain, before any agent behavior gets added. You're building the *smallest* correct piece first, on purpose. **Read first:** [05_langchain_fundamentals Core Concepts](#) — "LCEL: connecting pieces with `|`".

What to do: 1. Pick one specialist to start with — the Writer is the easiest, since it needs no tools: given research notes (fake ones you make up are fine for now), it produces a draft. 2. Build it as one plain LCEL chain: `prompt | ChatOpenAI | output_parser`. No agent loop, no other specialists exist yet. 3. Test it directly against 2-3 made-up "research notes" inputs, read the drafts, and confirm the chain itself works well.

What's new (this is the start): you now have one working, tested piece of logic. **What stays the same going into Step 2:** this exact chain — you're not rewriting it, just wrapping it. **When you'll hit this for real:** every specialist agent in every multi-agent system you'll ever build starts exactly this way — prove the core logic alone before it has to answer to anyone else.

Your files after Step 1:

```
project_4_multi_agent_system/
├── agents/
│   └── writer_chain.py    → build_writer_chain() -> Runnable, plain LCEL, no ag
└── main.py               → calls the chain directly with made-up test notes
```

Step 2 — That Specialist, Now a Real Agent That Can Improve Its Own Work

Project: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — Step 2 of 5: That Specialist, Now a Real Agent That Can Improve Its Own Work

What this step does: takes Step 1's Writer chain and proves that *one* specialist can work as a full, standalone agent (with the ability to improve its own draft), before any teammates exist — keeping "does this one agent work" separate from "does the team work," so bugs are easy to trace. **Read first:** [07_ai_agents Core Concepts](#) — "The think → act → observe loop", [11_multi_agent_systems Core Concepts](#) — "Generator → Critic".

What to do: 1. Wrap Step 1's chain as `writer_agent.py`, with a real function:

`run_writer(notes, feedback=None) -> Draft` — `feedback` is what makes this an *agent that can improve*, not just a one-shot chain. 2. Build a small, standalone Reviewer as a second simple chain (no tools either) that scores a draft and gives back feedback text. 3. Connect a small, limited loop: Writer drafts → Reviewer scores → if below a bar, Writer redrafts using the feedback → repeat up to N times. 4. Test it: confirm the loop actually improves a first draft you made deliberately weak, and confirm it stops at the limit on a tricky case.

What's new vs. Step 1: the Writer chain gets a `feedback` option and a loop calling it; a second agent (Reviewer) now exists. **What stays the same:** the Writer's core logic from Step 1 is *reused*, not rewritten — you're wrapping it, not redoing it. **When you'll hit this for real:** any time output quality matters enough to justify a second pass — first drafts of anything (code, writing, analysis) are rarely the best version.

Your files after Step 2:

```
project_4_multi_agent_system/
├── agents/
│   ├── writer_agent.py      → run_writer(notes, feedback=None) -> Draft (wraps
│   └── reviewer_agent.py    → run_reviewer(draft) -> ReviewVerdict
├── main.py                  → limited Writer↔Reviewer loop, no supervisor y
└── test_writer_reviewer.py
```

Step 3 — A Research Agent That Looks Things Up Before Anyone Else Works

Project: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — Step 3 of 5: A Research Agent That Looks Things Up Before Anyone Else Works

What this step does: adds the specialist that starts the whole pipeline — proving it works on its own, exactly like Writer was proven in Steps 1-2, before it has a Supervisor or teammates depending on it. **What's new vs. Step 2:** a new Research agent exists, built and tested on its own. **What stays the same:** Writer and Reviewer from Steps 1-2 don't change. **When you'll hit this for real:** any pipeline where the first step is "go find out what's actually true" before anything downstream can be trusted — this is the pattern behind that. **Read first:** [06_tools_function_calling Core Concepts](#) (Research needs a real or fake lookup tool).

What to do: 1. Build the Research agent the same way you built Writer in Steps 1-2: a simple chain first if possible, turned into an agent with a lookup tool, since Research genuinely needs one (reuse Doc06's patterns). 2. Test it on its own against 2-3 made-up topics — confirm it produces usable research notes, before anything else depends on it.

Your files after Step 3:

```
project_4_multi_agent_system/
├── agents/
│   ├── writer_agent.py
│   ├── reviewer_agent.py
│   └── research_agent.py    → run_research(topic) -> ResearchNotes, tested on
└── main.py
```

Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings

Project: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — Step 4 of 5: An Analysis Agent That Turns Raw Research Into Organized Findings

What this step does: adds the specialist that sits between Research and Writer — proving it works on Step 3's real output, before the full pipeline exists. **What's new vs. Step 3:** a new Analysis agent takes in Research's output and produces organized findings. **What stays the same:** Research, Writer, and Reviewer are all untouched — Step 4 just adds something new that uses Research's output. **When you'll hit this for real:** any time raw information (research, logs, survey results) needs organizing into a usable shape before someone (or something) can write from it — a very common real pipeline stage. **Read first:** [05_langchain_fundamentals Core Concepts](#) (Analysis is likely just a plain chain, no tools needed).

What to do: 1. Build the Analysis agent: takes Step 3's `ResearchNotes`, produces organized findings the Writer can draft from. 2. Test it on its own: feed it Step 3's real test outputs (not new made-up data) — this confirms the two specialists' interfaces genuinely work together, before the Supervisor exists to connect them.

Your files after Step 4:

```
project_4_multi_agent_system/
├── agents/
│   ├── writer_agent.py
│   ├── reviewer_agent.py
│   ├── research_agent.py
│   └── analysis_agent.py      → run_analysis(notes) -> Findings, tested again
└── main.py
```

Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command (final: 5 agents)

Project: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — Step 5 of 5: The Full Team: a Supervisor Routing 4 Specialists via Command

What this step does: adds the Supervisor that routes work on the fly using the `Command` tool — turning four separately-proven specialists into one connected pipeline. **What's new vs. Step 4:** a Supervisor now handles routing (replacing any standalone test loops); shared state is properly designed; Step 2's limited Writer ↔ Reviewer loop gets reconnected *inside* this bigger flow. **What stays the same:** all four specialists' inner logic is *unchanged* from Steps 1-4 — they're now called *by* the Supervisor instead of by your test scripts, but their own code doesn't need to change to join the team. This is the whole point of building it in this order: putting pieces together, not rewriting them. **When you'll hit this for real:** this exact moment — separately-proven pieces getting composed into one system by a coordinator — is what "building a multi-agent system" actually means at a real job, more than any single agent's internal logic. **Read first:** [11_multi_agent_systems Core Concepts — full pattern table + "The `Command` tool"](#).

What to do: 1. Design the shared state shape — clear about what's shared (the growing brief) vs. private to each agent (each specialist's own working notes), following Doc11's warning about notes leaking between agents. 2. Build the Supervisor node: routes to Research → Analysis → Writer → Reviewer in order, using `Command`, keeping track of what's already been done so no agent redundantly repeats another's work. 3. Reconnect Step 2's limited Writer ↔ Reviewer loop *inside* this bigger flow — same limit, same logic, now started by the Supervisor. 4. Test the full pipeline start to finish, plus: a task where the Supervisor has to route correctly, and the tricky non-agreement case from Step 2, now inside the full system.

Your files after Step 5 (final):

```
project_4_multi_agent_system/
├─ main.py
├─ graph.py
├─ state.py
├─ agents/
│   ├─ supervisor.py           → routing logic using Command
│   ├─ research_agent.py       → unchanged from Step 3
│   ├─ analysis_agent.py       → unchanged from Step 4
│   ├─ writer_agent.py         → unchanged from Step 2
│   └─ reviewer_agent.py       → unchanged from Step 2
└─ test_project4.py
```

Final Deliverable: ContentForge-Multi-Agent-Turn-Any-Topic-Into-A-Published-Brief — a Supervisor that routes Research, Analysis, Writer, and Reviewer on the fly via `Command`, with a carefully-designed shared state and a limited revision loop, turning a raw topic into a fact-checked final brief.

Checklist Before You Call This Done

- [] Supervisor routes correctly using the `Command` tool, not a hand-made routing text code
- [] Shared state is deliberately organized — no unfiltered mixing of notes between agents
- [] Reviewer can reject and send work back to the writer, with a fixed limit on revision rounds
- [] No repeated tool calls across agents for the same sub-task
- [] The full task completes through all four agents in a real test case
- [] You can defend "why 4 agents and not 1" without being asked twice, backed by your Doc11 cost/speed comparison

Full requirements, test cases, and hints: [11_multi_agent_systems/README.md](#).

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** through **Hint 4** about the exact part you're stuck on. Only ask for the full code if you say "**Show me the solution.**"